

# COLSON: Controllable Learning-Based Social Navigation via Diffusion-Based Reinforcement Learning

Yuki Tomita<sup>1†</sup>, Kohei Matsumoto<sup>2†</sup>, Yuki Hyodo<sup>1</sup>, and Ryo Kurazume<sup>2</sup>

**Abstract**—Mobile robot navigation in dynamic environments with pedestrian traffic is a key challenge in the development of autonomous mobile service robots. Recently, deep reinforcement learning-based methods have been actively studied and have outperformed traditional rule-based approaches owing to their optimization capabilities. Among these, methods that assume a continuous action space typically rely on a Gaussian distribution assumption, which limits the flexibility of generated actions. Meanwhile, the application of diffusion models to reinforcement learning has advanced, allowing for more flexible action distributions compared with Gaussian distribution-based approaches. In this study, we applied a diffusion-based reinforcement learning approach to social navigation and validated its effectiveness. Furthermore, by leveraging the characteristics of diffusion models, we propose an extension that enables post-training action smoothing and adaptation to static obstacle scenarios not considered during the training steps.

## I. INTRODUCTION

Social navigation, which enables safe and efficient navigation in dynamic environments with pedestrian traffic, is crucial for realizing autonomous mobile service robots. In recent years, driven by advancements in machine learning, deep reinforcement learning-based methods have been studied extensively. Among these, methods that assume a continuous action space require modeling of the action distribution using predefined functions, such as Gaussian distributions. Generative models, a branch of machine learning, have seen remarkable advancements, with diffusion models achieving significant success in fields such as image generation. Their application to reinforcement learning has also been explored, and they can reportedly achieve a high performance in continuous action spaces owing to their expressive capacity and lack of reliance on a Gaussian distribution assumption. However, diffusion models have not yet been applied to social navigation. Therefore, this study applied a diffusion-based reinforcement learning approach to mobile robot navigation in dynamic environments with pedestrian traffic.

In addition, many deep reinforcement learning-based methods face challenges adapting to conditions that are not considered during training. For instance, methods trained in environments with only pedestrians struggle when deployed

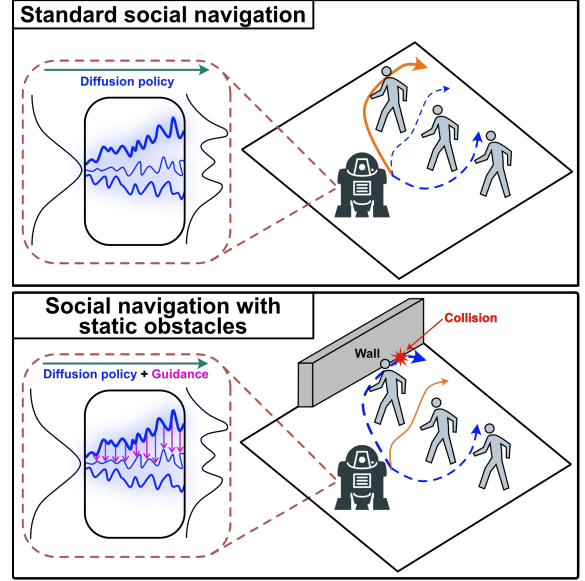


Fig. 1. Conceptual diagram of the proposed method. In standard social navigation, the proposed method leverages the multimodality of diffusion models to generate various actions that allow the robot to avoid pedestrians. The blue dashed arrows represent examples of candidate actions generated for this situation, while the solid orange line indicates the actual action selected (upper window). In environments with static obstacles, actions generated solely by the diffusion model may result in collisions with walls. Therefore, guidance is applied to steer the selection toward alternative candidates that avoid wall collisions (lower window).

in environments with static obstacles, such as walls. This study aimed to address this issue by utilizing guidance, a technique that allows the incorporation of post-training constraints during action generation using diffusion models without additional training. Considering these adaptability features, the proposed method is called **Controllable Learning-based Social Navigation (COLSON)**. A conceptual diagram of COLSON is shown in Fig. 1.

The main contributions of this study are as follows:

- We integrate a diffusion-based reinforcement learning method with a graph neural network (GNN) architecture and apply it to social navigation.
- We demonstrate that the proposed method can handle conditions that are not considered during training, such as trajectory smoothing and adaptation to environments with static obstacles, without requiring additional training.
- Through comparative experiments with conventional methods, we show that the proposed method outperforms existing approaches and exhibits high performance, even in environments where the number of

\*This work was not supported by any organization

<sup>1</sup>Yuki Tomita and Yuki Hyodo are with the Graduate School of Information Science and Electrical Engineering, Kyushu University, Fukuoka 819-0395, Japan [tomita@irvs.ait.kyushu-u.ac.jp](mailto:tomita@irvs.ait.kyushu-u.ac.jp), [hyodo@irvs.ait.kyushu-u.ac.jp](mailto:hyodo@irvs.ait.kyushu-u.ac.jp)

<sup>2</sup>Kohei Matsumoto and Ryo Kurazume are with the Faculty of Information Science and Electrical Engineering, Kyushu University, Fukuoka 819-0395, Japan [matsumoto@irvs.ait.kyushu-u.ac.jp](mailto:matsumoto@irvs.ait.kyushu-u.ac.jp), [kurazume@ait.kyushu-u.ac.jp](mailto:kurazume@ait.kyushu-u.ac.jp)

<sup>†</sup>Authors contributed equally

pedestrians differs from that in the training phase, highlighting its scalability.

- We conducted a real-world demonstration to validate that the proposed method can effectively control an actual robot in practical situations.

## II. RELATED WORK

### A. Social Navigation

Numerous deep reinforcement learning-based social navigation methods have been proposed. Chen et al. [1] introduced value-based learning models for multi-agent collision avoidance, marking a pioneering achievement in deep reinforcement learning approaches aimed at enabling autonomous mobile robots to safely navigate dynamic environments. Building on this, Chen et al. [2] proposed a method that incorporates social norms into the reward function to improve pedestrian collision avoidance and further addressed the challenge of pedestrian switching in multi-agent scenarios by sharing weights in the pedestrian information processing layer and employing max pooling. In addition to the direct application of deep reinforcement learning, architectural improvements in neural networks have been explored to enhance performance and develop methods that can accommodate an arbitrary number of agents [3], [4], [5], [6], [7], [8]. Other studies have expanded the scope beyond performance enhancement and scalability to pedestrian density by investigating approaches that leverage human gaze information [9], utilize map-based processing to enable navigation in environments with both static and dynamic obstacles [10], [11], and incorporate continuous action spaces [12]. More recently, hybrid approaches that integrate reinforcement learning-based methods with rule-based methods have been proposed to further enhance robustness and adaptability [13], [14].

### B. Diffusion Models for Action Generation

Diffusion models have achieved remarkable success in various data generation tasks, including image synthesis [15], [16], [17]. Recent studies have extended their applications beyond vision-based data to action generation in robotics through imitation learning and reinforcement learning. In the field of imitation learning, Janner et al. and Reuss et al. proposed methods based on behavior cloning (BC) [18], [19], where diffusion models were trained to generate actions that replicate target demonstration data, making it the most fundamental approach for applying diffusion models to action generation. In addition to imitation learning, diffusion models have been applied to offline reinforcement learning [20], [21], [22], [23], where exploration is not required, allowing models to learn distributions from datasets in a manner similar to that of imitation learning.

More recently, efforts have been directed toward applying diffusion models to online reinforcement learning [24], particularly in settings that differ from conventional diffusion model formulations, such as learning without access to pre-sampled data from an optimal target distribution [25] and approaches based on weighted regression [26].

Furthermore, in the context of applying diffusion models to mobile robots, methods leveraging diffusion policy for goal-conditioned navigation and undirected exploration have been proposed [27]. Other approaches use models trained on ground truth data from the A\* algorithm for global path planning [28], [29].

## III. PRELIMINARIES

This study addresses the problem of two-dimensional (2D) mobile robot navigation while ensuring collision avoidance with pedestrians. The robot's state includes its position from the goal, velocity, and orientation information:  $\mathbf{s}^r = [^g\mathbf{p}^r, \mathbf{v}^r, \theta]$ . By contrast, the position and velocity information of each pedestrian is converted to a robot-centered coordinate system and adopted as a state  $\mathbf{s}^n = [^{rc}\mathbf{p}^n, ^{rc}\mathbf{v}^n]$ . The set of pedestrian states in the environment is described by  $\mathbf{s}^h = [\mathbf{s}^1, \mathbf{s}^2, \dots, \mathbf{s}^n]$ . The robot obtains rewards after performing actions.  $R_t$  denotes the reward function at time  $t$  as follows:

$$R_t = \begin{cases} -0.25 & \text{if } d_t < 0 \\ -0.1 + d_t/2 & \text{else if } d_t < 0.2 \\ 1 & \text{else if } \mathbf{p}_t^r = \mathbf{p}_t^g \\ 0 & \text{otherwise} \end{cases}, \quad (1)$$

where  $d_t$  is the minimum separation distance between the robot and pedestrians,  $\mathbf{p}_t^r$  is the robot's position, and  $\mathbf{p}_t^g$  is the goal position at time  $t$ .

## IV. APPROACH

In this section, the model architecture, training methodology, and guidance methods for path smoothing and static obstacle avoidance are presented.

### A. Model Architecture

The model architecture of the proposed method is illustrated in Fig. 2. This model adopts an Actor-Critic framework

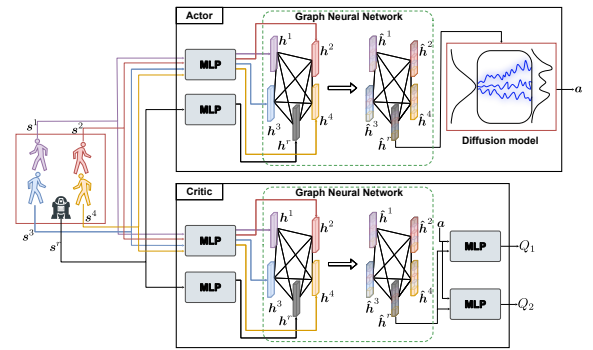


Fig. 2. Architecture of proposed method.  $\mathbf{s}^r$  and  $\mathbf{s}^n$  indicate the states of each robot and pedestrian, respectively.  $\mathbf{h}^n$  indicates the features of each robot and pedestrian.  $\hat{\mathbf{h}}^n$  indicates the features after the GNN is applied.

in which both the Actor and Critic incorporate GNNs. **Each GNN utilizes Graph Attention Networks v2 (GATv2) [30]. The Actor employs a diffusion model that considers the output of the GNN as a condition and generates actions.** The Critic network inputs the output of the GNN into a multi-layer perceptron to compute Q-values.

### B. Q-Score Matching

In this study, we used Q-score matching (QSM) [25] as the training framework. Diffusion models are typically trained using score functions for the target distribution. However, in reinforcement learning, where the objective is to learn an optimal policy, the optimal policy is unknown. Consequently, neither the score function nor samples from the optimal policy can be precomputed. To overcome this challenge, QSM facilitates reinforcement learning with diffusion models by leveraging the action gradient of the Q-function as a score. The training procedure using QSM is presented in Algorithm 1. The coefficient  $\alpha$  required for computing the

---

#### Algorithm 1: Training algorithm using QSM

---

```

1 Initialize score network  $\Psi_\theta$  and critic networks  $Q_{\phi 1}$  and  $Q_{\phi 2}$ 
2 Set parameter values of the target critics  $Q_{\phi',1}$  and  $Q_{\phi',2}$  equal to
  those of the main critics
3 for  $i = 1$  to  $E$  do
4   Explore using the policy until finishing an episode
5   After finishing an episode, store the trajectory of
      $(s_t, a_t, r_t, s'_t)$  to the replay buffer  $\mathcal{D}$ 
6   Sample a batch  $\mathcal{B} = \{(s, a, r, s')\}$  from the replay buffer  $\mathcal{D}$ 
7   Sample actions for computing targets  $\hat{a}' \sim \pi_\theta(\cdot | s')$ 
8   Calculate targets for the Q-function
      $y(r, s') = r + \gamma (\min_{i=1,2} Q_{\phi',i}(s', \hat{a}'))$ 
9   Update critics by minimizing
      $L_{\text{critic}} = \frac{1}{|\mathcal{B}|} \sum (Q_{\phi^i}(s, a) - y(r, s'))^2$  for  $i = 1, 2$ 
10  Update score network by minimizing
      $L_{\text{QSM}} = \frac{1}{|\mathcal{B}|} \sum \|\Psi_\theta(s, a) - \alpha \nabla_a Q(s, a)\|^2$ 
11  Update the target critic by polyak averaging
      $\phi^{i,i} \leftarrow \rho \phi^{i,i} + (1 - \rho) \phi_i$  for  $i = 1, 2$ 
12 end
```

---

loss function must be set by considering the possible values of the Q-function and the scale of the final actions. In this study, several configurations of  $\alpha$  were tested, and the best configuration was selected.

### C. Guidance for Smooth Action Generation

As an initial demonstration of the proposed method's extensibility, we show that action smoothing can be achieved during generation without additional training by introducing additional constraints. The reward function in this study prioritizes reaching the goal and avoiding collisions but does not account for trajectory smoothness. Consequently, the generated actions often exhibit abrupt movements. To address this issue, we employ Stochastic Differential Editing (SDEdit) [31] for smoothing. Diffusion models generate outputs by progressively removing Gaussian noise. SDEdit modifies this process by initializing it with reference data augmented with noise, guiding the generated data to approximate the desired output. In our method, the actions from the previous step serve as the guide data. The execution procedure for smoothing with SDEdit is presented in Algorithm 2.

### D. Guidance for Static Obstacle Avoidance

As a second demonstration of the proposed method's extensibility, we show that guidance enables the robot to

navigate around static obstacles that were absent during training without further training. The training environment

---

#### Algorithm 2: Action generation with guidance for smoothing

---

```

Input: Trained score network  $\Psi_\theta$ , states  $s_t = [s_t^r, s_t^h]$  at time  $t$ 
         Previous action  $a_{t-1}$ , hyperparameter  $T, \alpha_\tau$ 
1  $z = \mathcal{N}(0, I)$ 
2  $x_0 = a_{t-1}$ 
3 for  $\tau = 1$  to  $T$  do
4    $x_\tau = \sqrt{1 - \alpha_\tau} x_{\tau-1} + \sqrt{\alpha_\tau} z$ 
5 end
6 for  $\tau = T$  to 1 do
7   Denoise by score employing
      $x_{\tau-1} = \frac{1}{\sqrt{1 - \alpha_\tau}} x_\tau - \frac{\sqrt{\alpha_\tau}}{\sqrt{1 - \alpha_\tau}} \Psi_\theta(s_t, x_\tau, \tau)$ 
8 end
9 Execute the action  $a_t = x_0$ 
```

---

in this study contained only pedestrians, implying that the learned policy lacks the capability to avoid static obstacles.

---

#### Algorithm 3: Action generation with guidance for static obstacle avoidance

---

```

Input: Trained score network  $\Psi_\theta$ , states  $s_t = [s_t^r, s_t^h]$  at time  $t$ ,
         previous action  $a_{t-1}$ , current robot position  $p^r$ , safety
         distance  $d_s$ , hyperparameter  $\eta, T, \alpha_\tau$ , distance cost  $c_d$ ,
         lethal cost  $c_l$ 
1  $x_T = \mathcal{N}(0, I)$ 
2 for  $\tau = T$  to 1 do
3   Calculate next robot position  $\hat{p}^r = p^r + x_\tau \Delta t$  and
     difference from the nearest point of static obstacles  $d$ 
4   Calculate vector for guidance to avoid static obstacles
5    $g_o = \begin{cases} \frac{d}{|d|} c_l & \text{if } \hat{p}^r \text{ is inside of obstacles} \\ \frac{d}{|d|} c_d & \text{else if } 0 < |d| \leq d_s \\ \frac{d}{|d|} c_d \exp(\eta(d_s - |d|)) & \text{otherwise} \end{cases}$ 
6   Denoise by score and guidance employing
      $x_{\tau-1} = \frac{1}{\sqrt{1 - \alpha_\tau}} x_\tau - \frac{\sqrt{\alpha_\tau}}{\sqrt{1 - \alpha_\tau}} \Psi_\theta(s_t, x_\tau, \tau) + g_o$ 
7 end
8 Execute the action  $a_t = x_0$ 
```

---

To address this, we incorporate guidance into the diffusion model to steer actions away from static obstacles, allowing the robot to avoid both pedestrians and static obstacles. In particular, the guidance strength increases as the robot approaches a wall, directing its movement away from the obstacle. The execution procedure for this guidance is presented in Algorithm 3.

## V. EXPERIMENTS

Experiments were conducted in a simulation environment to evaluate the effectiveness of the proposed method.

### A. Simulation Environment and Settings

We employed the CrowdNav environment adopted in some related studies [4], [5]. In this environment, pedestrians are controlled by ORCA [32]. The models were trained for 100,000 episodes in a visible circle crossing scenario with 5 pedestrians. Additionally, data were collected using ORCA before each training session, and training began with a dataset of 2000 episodes. The hyperparameter  $\alpha$  of QSM

TABLE I  
NUMERICAL COMPARISON IN CIRCLE-CROSSING SCENARIO WITH 5 ORCA PEDESTRIANS

| Method               | Visible             |                    |                    |                      | Invisible           |                    |                    |                      |
|----------------------|---------------------|--------------------|--------------------|----------------------|---------------------|--------------------|--------------------|----------------------|
|                      | Success [%] ↑       | Collision [%] ↓    | Exec. time [s] ↓   | Return ↑             | Success [%] ↑       | Collision [%] ↓    | Exec. time [s] ↓   | Return ↑             |
| ORCA (for reference) | 100.00 ± 0.00       | 0.00 ± 0.00        | 10.02 ± 0.00       | 0.542 ± 0.000        | 42.80 ± 0.00        | 56.80 ± 0.00       | 10.93 ± 0.00       | 0.081 ± 0.000        |
| BC                   | 77.28 ± 0.17        | 22.56 ± 0.18       | 10.06 ± 0.61       | 0.377 ± 0.001        | 5.08 ± 0.21         | 94.92 ± 0.21       | 10.24 ± 0.81       | -0.182 ± 0.002       |
| AWR                  | 87.60 ± 1.96        | 12.36 ± 1.97       | 11.44 ± 1.80       | 0.466 ± 0.009        | 42.44 ± 4.78        | 57.56 ± 4.78       | 11.48 ± 2.16       | 0.122 ± 0.030        |
| CAWR                 | 61.00 ± 0.57        | 35.28 ± 0.45       | 11.98 ± 8.13       | 0.236 ± 0.001        | 12.48 ± 2.11        | 86.68 ± 2.34       | 12.96 ± 10.21      | 0.122 ± 0.030        |
| QVWR                 | 62.48 ± 3.84        | 34.24 ± 3.86       | 11.44 ± 5.63       | 0.245 ± 0.020        | 6.68 ± 1.00         | 92.92 ± 1.10       | 11.44 ± 6.04       | -0.173 ± 0.007       |
| AWAC                 | 71.08 ± 2.91        | 28.92 ± 2.91       | 11.32 ± 5.83       | 0.327 ± 0.011        | 29.76 ± 7.20        | 70.24 ± 7.20       | 11.31 ± 5.69       | 0.010 ± 0.038        |
| DIPO                 | 80.80 ± 1.97        | 17.52 ± 1.42       | 11.29 ± 1.26       | 0.327 ± 0.011        | 30.32 ± 3.41        | 68.96 ± 3.03       | 12.22 ± 0.15       | 0.022 ± 0.021        |
| QVPO                 | 98.24 ± 0.02        | 1.68 ± 0.01        | 9.75 ± 0.97        | 0.618 ± 0.001        | 97.44 ± 0.03        | 2.36 ± 0.03        | 9.99 ± 0.79        | 0.606 ± 0.001        |
| COLSON (QSM)         | <b>98.28 ± 0.03</b> | <b>1.60 ± 0.03</b> | <b>8.70 ± 0.07</b> | <b>0.647 ± 0.000</b> | <b>98.44 ± 0.01</b> | <b>1.32 ± 0.01</b> | <b>9.05 ± 0.05</b> | <b>0.640 ± 0.000</b> |

TABLE II  
NUMERICAL COMPARISON IN CIRCLE-CROSSING SCENARIO WITH 5 SOCIAL FORCE PEDESTRIANS

| Method               | Visible             |                    |                    |                      | Invisible           |                    |                    |                      |
|----------------------|---------------------|--------------------|--------------------|----------------------|---------------------|--------------------|--------------------|----------------------|
|                      | Success [%] ↑       | Collision [%] ↓    | Exec. time [s] ↓   | Return ↑             | Success [%] ↑       | Collision [%] ↓    | Exec. time [s] ↓   | Return ↑             |
| ORCA (for reference) | 90.80 ± 0.00        | 9.20 ± 0.00        | 9.79 ± 0.00        | 0.544 ± 0.000        | 19.00 ± 0.00        | 81.00 ± 0.00       | 10.82 ± 0.00       | -0.088 ± 0.000       |
| BC                   | 63.4 ± 1.30         | 36.6 ± 1.30        | 10.16 ± 0.73       | 0.303 ± 0.007        | 0.28 ± 0.00         | 99.72 ± 0.00       | 14.62 ± 75.05      | -0.222 ± 0.000       |
| AWR                  | 86.92 ± 0.57        | 13.04 ± 0.58       | 11.52 ± 2.18       | 0.483 ± 0.007        | 41.68 ± 11.13       | 58.28 ± 11.16      | 11.37 ± 2.00       | 0.131 ± 0.076        |
| CAWR                 | 63.32 ± 1.94        | 34.20 ± 2.25       | 12.28 ± 11.09      | 0.278 ± 0.005        | 6.80 ± 0.90         | 92.48 ± 1.12       | 21.17 ± 70.54      | -0.172 ± 0.006       |
| QVWR                 | 58.04 ± 0.89        | 36.36 ± 1.91       | 12.46 ± 7.28       | 0.245 ± 0.003        | 3.60 ± 0.33         | 96.08 ± 0.38       | 23.85 ± 70.94      | -0.196 ± 0.003       |
| AWAC                 | 80.64 ± 3.24        | 19.32 ± 3.25       | 11.47 ± 7.14       | 0.431 ± 0.022        | 42.68 ± 6.67        | 57.32 ± 6.67       | 15.53 ± 70.50      | 0.127 ± 0.048        |
| DIPO                 | 73.52 ± 0.41        | 25.00 ± 0.24       | 12.06 ± 1.30       | 0.362 ± 0.001        | 6.88 ± 0.63         | 92.32 ± 0.54       | 12.80 ± 0.72       | -0.154 ± 0.004       |
| QVPO                 | <b>99.88 ± 0.00</b> | <b>0.04 ± 0.00</b> | <b>9.47 ± 1.08</b> | <b>0.642 ± 0.001</b> | <b>96.72 ± 0.08</b> | <b>3.20 ± 0.08</b> | <b>9.76 ± 0.99</b> | <b>0.607 ± 0.002</b> |
| COLSON (QSM)         | 99.60 ± 0.00        | 0.40 ± 0.00        | <b>8.40 ± 0.03</b> | <b>0.669 ± 0.000</b> | <b>98.72 ± 0.00</b> | <b>1.20 ± 0.00</b> | <b>8.62 ± 0.07</b> | <b>0.654 ± 0.000</b> |

was set to 400. The number of generation steps was set to 100 for each diffusion-based reinforcement learning method.

### B. Performance Evaluation

The performance of each method was evaluated in a circle crossing scenario with pedestrians controlled by ORCA and another scenario with pedestrians controlled by the social force model. In each scenario, the methods was evaluated under two conditions: one where pedestrians were aware of the robot's position (visible) and another where they were unaware (invisible). For comparison, the proposed method was evaluated against BC, CAWR [12], which tackles social navigation with continuous action spaces, and related reinforcement learning methods, such as AWR [33], QVWR [34], and AWAC [35]. Furthermore, comparisons were made with diffusion model-based reinforcement learning methods, including DIPO [24] and QVPO [26]. For each method, five models were trained with different random seeds. The evaluation metrics included success rate, collision rate, average execution time, and average return. The test scenarios comprised 500 episodes each.

The results are summarized in Tables I and II. The ± symbol in each table indicates the standard deviation computed from the results of training runs using five independent random seeds. In scenarios where pedestrians were controlled using ORCA, the proposed method achieved the best performance under both the visible and invisible conditions. In scenarios where pedestrians were controlled using the social force model, QVPO achieved better results in terms of success and collision rates under the visible condition. However, the proposed method executed approximately 1 s faster and achieved higher returns. In the invisible setting, the proposed method outperformed all categories.

Fig. 3 shows a comparison of success rates as the number of pedestrians increases in the circle crossing scenario. The proposed method consistently outperformed the other

approaches across all pedestrian densities. The performance gap with QVPO is small for five pedestrians but widens as the population increases. This trend is likely due to QVPO's reliance on weighted regression, which limits generalization to unseen scenarios.

### C. Evaluation of Smoothing

The effectiveness of smoothing through guidance using SDEdit was evaluated in the circle crossing scenario with 20 pedestrians, where action oscillations were prominent. Fig. 4 presents a qualitative comparison, showing that trajectories generated with SDEdit-based smoothing are noticeably smoother. In addition, Table III provides a quantitative comparison of success rate and trajectory smoothness. Trajectory smoothness was evaluated using a measure from a previous study [36]. The numerical evaluation results indicate that although SDEdit reduces the success rate, it significantly enhances trajectory smoothness. Despite a slight decrease in success rate, reduced execution time results in a higher return. This is likely because improved smoothness allows surrounding pedestrians to avoid the robot more easily.

TABLE III  
NUMERICAL COMPARISON FOR SMOOTHING

| Method       | Success [%] ↑ | Collision [%] ↓ | Exec. time [s] ↓ | Return ↑     | Smoothness · 10 <sup>3</sup> ↓ |
|--------------|---------------|-----------------|------------------|--------------|--------------------------------|
| w/o guidance | <b>82.60</b>  | <b>17.20</b>    | 11.35            | 0.445        | 43.09                          |
| w guidance   | 82.00         | 17.40           | <b>10.97</b>     | <b>0.448</b> | <b>9.16</b>                    |

### D. Evaluation of Static Obstacle Avoidance

The effectiveness of the guidance for static obstacle avoidance was evaluated in an environment with walls and three pedestrians approaching from the front. Fig. 5 presents a qualitative comparison. The results show that when the robot initially intended to move left to avoid pedestrians, leading to a potential collision with the wall, shifting to the right by the guidance instead allowed it to evade both pedestrians and the wall. Table IV presents a quantitative comparison of cases with and without guidance. In the table, Col-P represents

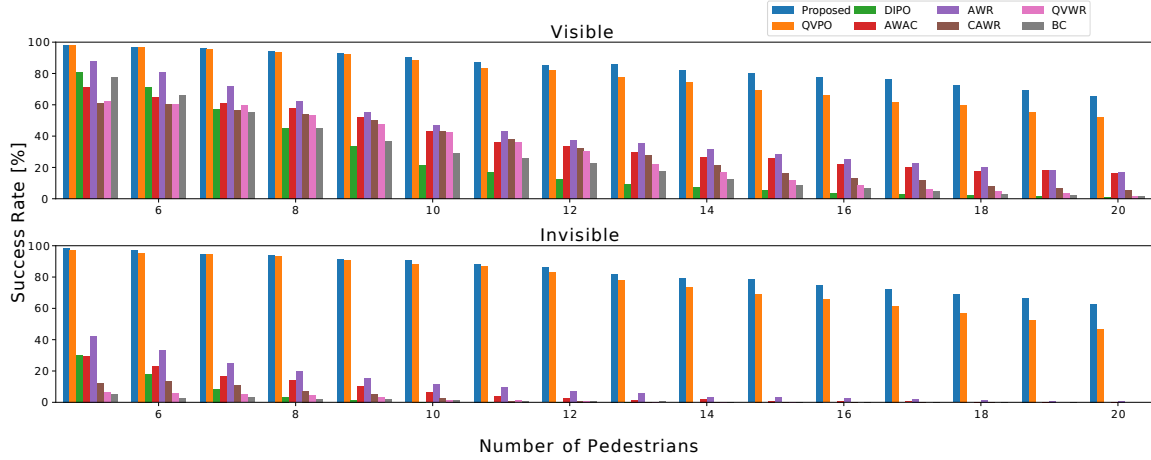


Fig. 3. Comparison of success rate while changing the number of pedestrians. The upper graph shows the results for the visible setting, while the lower graph presents the results for the invisible setting.

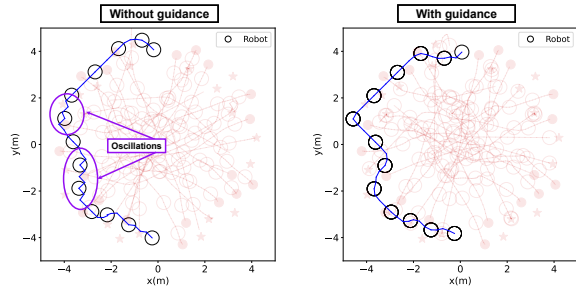


Fig. 4. Trajectory of robot with and without guidance for smoothing. The blue lines represent the trajectory of the robot's movement, while the light red lines indicate the trajectories of pedestrian movements.

the collision rate with pedestrians, and Col-W represents the collision rate with walls. The results show that the proposed guidance method significantly reduces collisions with walls and greatly improves the success rate.

TABLE IV

NUMERICAL COMPARISON IN ENVIRONMENTS WITH STATIC OBSTACLES

| Method       | Success [%]↑ | Col-P [%]↓ | Col-W [%]↓ | Exec. time [s]↓ | Return↑      |
|--------------|--------------|------------|------------|-----------------|--------------|
| w/o guidance | 52.8         | 0.04       | 46.8       | 8.00            | 0.261        |
| w guidance   | <b>95.6</b>  | 0.08       | <b>3.6</b> | <b>7.99</b>     | <b>0.645</b> |

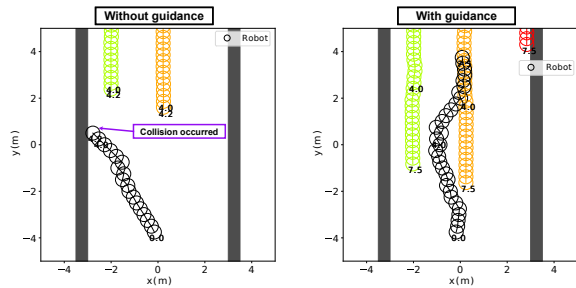


Fig. 5. Comparison between with and without guidance for static obstacle avoidance. Colored circles represent pedestrian trajectories, and the black rectangles represent walls.

## VI. REAL-WORLD DEMONSTRATION

A real-world demonstration was conducted using the developed robot system. The system is based on a Mecanum rover and is equipped with a 2D-LiDAR (UST-20LX) and

onboard computer (Jetson AGX Orin) for processing. The software system was built using ROS2 Humble, and pedestrian detection was performed using DR-SPAAM [37], [38]. Additionally, localization was realized using adaptive Monte Carlo localization (AMCL). The demonstration setup is

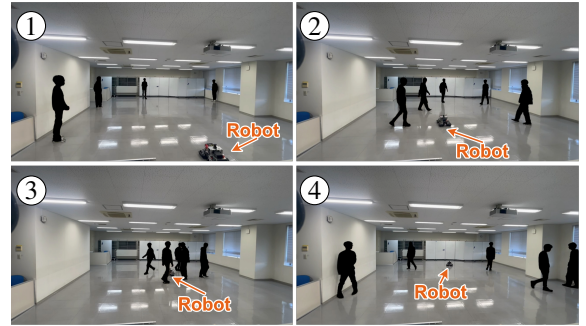


Fig. 6. Scenes of the real-world demonstration using the proposed method.

shown in Fig. 6. The results confirm that the proposed method is capable of controlling a real robot.

Meanwhile, the average action generation time for the proposed method was 0.28 s, with a maximum of 0.41 s with the robot system. Given that the training environment used a 0.25 s time interval, further speed optimization is necessary. Future studies should address this issue.

## VII. CONCLUSIONS

In this study, we applied diffusion-based reinforcement learning trained with QSM for social navigation and demonstrated that it outperforms other methods. In addition, we demonstrated that by utilizing guidance, conditions that were not considered during training could be incorporated without additional training. In future, we plan to extend the applicability of the proposed method to a wider range of scenarios and conduct more detailed real-world experiments to further enhance its performance.

## REFERENCES

- [1] Y. F. Chen, M. Liu, M. Everett, and J. P. How, "Decentralized Non-communicating Multiagent Collision Avoidance with Deep Reinforce-



- ment Learning,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pp. 285–292, 2017.
- [2] Y. F. Chen, M. Everett, M. Liu, and J. P. How, “Socially Aware Motion Planning with Deep Reinforcement Learning,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 1343–1350, 2017.
  - [3] M. Everett, Y. F. Chen, and J. P. How, “Motion Planning among Dynamic, Decision-Making Agents with Deep Reinforcement Learning,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 3052–3059, 2018.
  - [4] C. Chen, Y. Liu, S. Kreiss, and A. Alahi, “Crowd-Robot Interaction: Crowd-aware Robot Navigation with Attention-based Deep Reinforcement Learning,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pp. 6015–6022, 2019.
  - [5] C. Chen, S. Hu, P. Nikdel, G. Mori, and M. Savva, “Relational Graph Learning for Crowd Navigation,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 10007–10013, 2020.
  - [6] K. Matsumoto, A. Kawamura, Q. An, and R. Kurazume, “Mobile Robot Navigation Using Learning-Based Method Based on Predictive State Representation in a Dynamic Environment,” in *Proceedings of the IEEE/SICE International Symposium on System Integration (SII)*, pp. 499–504, 2022.
  - [7] Y. Yang, J. Jiang, J. Zhang, J. Huang, and M. Gao, “ST<sup>2</sup>: Spatial-Temporal state transformer for Crowd-Aware autonomous navigation,” *IEEE Robotics and Automation Letters*, vol. 8, no. 2, pp. 912–919, 2023.
  - [8] S. Liu, P. Chang, Z. Huang, N. Chakraborty, K. Hong, W. Liang, D. Livingston McPherson, J. Geng, and K. Driggs-Campbell, “Intention Aware Robot Crowd Navigation with Attention-Based Interaction Graph,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pp. 12015–12021, 2023.
  - [9] Y. Chen, C. Liu, B. E. Shi, and M. Liu, “Robot Navigation in Crowds by Graph Convolutional Networks With Attention Learned From Human Gaze,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 2754–2761, 2020.
  - [10] L. Lucia, D. Daniel, C. Gianluca, S. Roland, and D. Renaud, “Robot Navigation in Crowded Environments Using Deep Reinforcement Learning,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 5671–5677, 2020.
  - [11] S. Yao, G. Chen, Q. Qiu, J. Ma, X. Chen, and J. Ji, “Crowd-Aware Robot Navigation for Pedestrians with Multiple Collision Avoidance Strategies via Map-based Deep Reinforcement Learning,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 8144–8150, 2021.
  - [12] X. Zhang, W. Xi, X. Guo, Y. Fang, B. Wang, W. Liu, and J. Hao, “Relational Navigation Learning in Continuous Action Space among Crowds,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pp. 3175–3181, 2021.
  - [13] J. Wu, Y. Wang, H. Asama, Q. An, and A. Yamashita, “Risk-Sensitive Mobile Robot Navigation in Crowded Environment via Offline Reinforcement Learning,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 7456–7462, 2023.
  - [14] K. Matsumoto, Y. Hyodo, and R. Kurazume, “Crowd-Aware Robot Navigation with Switching Between Learning-Based and Rule-Based Methods Using Normalizing Flows,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 4823–4830, 2024.
  - [15] J. Ho, A. Jain, and P. Abbeel, “Denoising Diffusion Probabilistic Models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 6840–6851, 2020.
  - [16] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-Resolution Image Synthesis with Latent Diffusion Models,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 10674–10685, 2022.
  - [17] K. Nakashima and R. Kurazume, “LiDAR Data Synthesis with Denoising Diffusion Probabilistic Models,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pp. 14724–14731, 2024.
  - [18] M. Janner, Y. Du, J. B. Tenenbaum, and S. Levine, “Planning with Diffusion for Flexible Behavior Synthesis,” in *Proceedings of the International Conference on Machine Learning (ICML)*, pp. 9902–9915, 2022.
  - [19] M. Reuss, M. Li, X. Jia, and R. Lioutikov, “Goal-Conditioned Imitation Learning using score-based Diffusion Policies,” in *Proceedings of the Robotics: Science and Systems (RSS)*, 2023.
  - [20] Z. Wang, J. J. Hunt, and M. Zhou, “Diffusion Policies as an Expressive Policy Class for Offline Reinforcement Learning,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.
  - [21] H. J. Terry Suh, G. Chou, H. Dai, L. Yang, A. Gupta, and R. Tedrake, “Fighting Uncertainty with Gradients: Offline Reinforcement Learning via Diffusion Score Matching,” in *Proceedings of the Annual Conference on Robot Learning (CoRL)*, pp. 2878–2904, 2023.
  - [22] B. Kang, X. Ma, C. Du, T. Pang, and Y. A. N. Shuicheng, “Efficient Diffusion Policies For Offline Reinforcement Learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 67195–67212, 2023.
  - [23] P. Hansen-Estruch, I. Kostrikov, M. Janner, J. G. Kuba, and S. Levine, “IDQL: Implicit Q-Learning as an Actor-Critic Method with Diffusion Policies,” *CoRR*, vol. abs/2304.10573, 2023.
  - [24] L. Yang, Z. Huang, F. Lei, Y. Zhong, Y. Yang, C. Fang, S. Wen, B. Zhou, and Z. Lin, “Policy Representation via Diffusion Probability Model for Reinforcement Learning,” *CoRR*, vol. abs/2305.13122, 2023.
  - [25] M. Psenka, A. Escontrela, P. Abbeel, and Y. Ma, “Learning a Diffusion Model Policy from Rewards via Q-Score Matching,” in *Proceedings of the International Conference on Machine Learning (ICML)*, pp. 41163–41182, 2024.
  - [26] S. Ding, K. Hu, Z. Zhang, K. Ren, W. Zhang, J. Yu, J. Wang, and Y. Shi, “Diffusion-based Reinforcement Learning via Q-weighted Variational Policy Optimization,” in *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 53945–53968, 2024.
  - [27] A. Sridhar, D. Shah, C. Glossop, and S. Levine, “NoMaD: Goal Masked Diffusion Policies for Navigation and Exploration,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pp. 63–70, 2024.
  - [28] J. Liu, M. Stamatopoulou, and D. Kanoulas, “DiPPeR: Diffusion-based 2D Path Planner applied on Legged Robots,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pp. 9264–9270, 2024.
  - [29] M. Stamatopoulou, J. Liu, and D. Kanoulas, “DiPPeST: Diffusion-based path planner for synthesizing trajectories applied on quadruped robots,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 7787–7793, 2024.
  - [30] S. Brody, U. Alon, and E. Yahav, “How Attentive are Graph Attention Networks?,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.
  - [31] C. Meng, Y. He, Y. Song, J. Song, J. Wu, J.-Y. Zhu, and S. Ermon, “SDEdit: Guided Image Synthesis and Editing with Stochastic Differential Equations,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.
  - [32] J. Van Den Berg, S. J. Guy, M. Lin, and D. Manocha, “Reciprocal n-Body Collision Avoidance,” in *Proceedings of the International Symposium of Robotic Research*, pp. 3–19, 2011.
  - [33] X. B. Peng, A. Kumar, G. Zhang, and S. Levine, “Advantage-Weighted Regression: Simple and Scalable Off-Policy Reinforcement Learning,” *CoRR*, vol. abs/1910.00177, 2019.
  - [34] P. Kozakowski, L. Kaiser, H. Michalewski, A. Mohiuddin, and K. Kańska, “Q-Value Weighted Regression: Reinforcement Learning with Limited Data,” in *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*, pp. 1–8, 2022.
  - [35] A. Nair, M. Dalal, A. Gupta, and S. Levine, “AWAC: Accelerating Online Reinforcement Learning with Offline Datasets,” *CoRR*, vol. abs/2006.09359, 2020.
  - [36] S. Mysore, B. Mabsout, R. Mancuso, and K. Saenko, “Regularizing Action Policies for Smooth Control with Reinforcement Learning,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pp. 1810–1816, 2021.
  - [37] D. Jia, A. Hermans, and B. Leibe, “DR-SPAAM: A Spatial-Attention and Auto-regressive Model for Person Detection in 2D Range Data,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 10270–10277, 2020.
  - [38] D. Jia, M. Steinweg, A. Hermans, and B. Leibe, “Self-Supervised Person Detection in 2D Range Data using a Calibrated Camera,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pp. 13301–13307, 2021.